Sumario

¿Qué es XML?....4
Características principales....4
¿Dónde se utiliza XML actualmente?....4
Configuración de aplicaciones....4
Intercambio de datos en sistemas empresariales....4
Bases de datos....5
Estándares internacionales....5
Documentos electrónicos....5
Industria del software y telecomunicaciones....5
Resumen didáctico....5
Instalación de herramientas para trabajar con XML (editores y validadores)....5
1. Editores de texto recomendados para XML....5
🖊️ Editores simples (para empezar)....5
💡 Editores avanzados (recomendados)....6
2. Validadores de XML....6
🔍 Validadores online....6
✅ Validadores locales (línea de comandos)....7
3. Instalación práctica paso a paso (Windows)....7
Instalar Visual Studio Code....7
Probar con un archivo XML simple....7
Validar online....7
4. Recomendación didáctica....8
Estructura básica de un documento XML....8
1. Prólogo (declaración inicial)....8
2. Elemento raíz....8
3. Elementos hijos y contenido....9
4. Atributos....9
5. Comentarios....9
6. Ejemplo completo de estructura básica....10
7. Reglas clave para un XML bien formado....10
8. Resumen didáctico....11
Elementos y etiquetas en XML....11
1. ¿Qué es una etiqueta en XML?....11
2. Tipos de elementos....11
a) Elementos simples....11
b) Elementos con hijos (anidados)....11
c) Elementos vacíos (self-closing)....12
3. Nombres de elementos....12
4. Jerarquía de elementos....12
5. Atributos vs. elementos....13
6. Ejemplo completo....13
7. Errores comunes con elementos y etiquetas....14
No cerrar una etiqueta....14
Cruzar etiquetas (anidamiento incorrecto)....14
Nombres inválidos en etiquetas....14
8. Resumen didáctico....14

Atributos en XML....14
Ejemplo básico....15
1. Reglas para los atributos....15
2. Diferencia entre atributos y elementos....15
3. Ejemplos prácticos....16
a) Información de un libro....16
b) Datos de un usuario....16
4. Casos donde conviene usar atributos....16
5. Casos donde conviene usar elementos....16
6. Ejemplo completo con atributos y elementos....17
7. Errores comunes con atributos....17
Olvidar las comillas....17
Usar el mismo atributo dos veces en un mismo elemento....17
Guardar datos complejos en atributos (mala práctica)....17
8. Resumen didáctico....18
Declaración XML y codificación de caracteres....18
1. ¿Qué es la declaración XML?....18
2. Atributos de la declaración XML....18
a) version....18
b) encoding....18
c) standalone....19
3. Importancia de la codificación....19
4. Ejemplos comparativos....19
a) Documento en UTF-8....19
b) Documento en ISO-8859-1....20
5. Buenas prácticas....20
6. Ejemplo completo....20
7. Resumen didáctico....20
Comentarios en XML....21
1. Sintaxis de un comentario....21
2. Ejemplos de comentarios....21
a) Comentario simple....21
b) Comentario en una línea dentro del contenido....21
c) Comentarios multilínea....21
3. Dónde se pueden colocar....22
4. Buenas prácticas con comentarios....22
5. Malas prácticas con comentarios....22
6. Ejemplo completo con comentarios....23
7. Resumen didáctico....23
Jerarquía y anidamiento de elementos en XML....23
1. Concepto de jerarquía en XML....23
2. Definiciones clave....24
3. Ejemplo básico de jerarquía....24
4. Representación en forma de árbol....24
5. Reglas de anidamiento correcto....24
6. Ejemplo de jerarquía más compleja....25
7. Ejemplo con atributos + elementos anidados....25
8. Buenas prácticas en jerarquía y anidamiento....26

9. Ejemplo de mala práctica....26
10. Resumen didáctico....26
Espacios de nombres (Namespaces) en XML....27
1. ¿Qué es un namespace en XML?....27
2. Problema que resuelven los namespaces....27
3. Sintaxis de namespaces....28
a) Declaración por defecto....28
b) Declaración con prefijo....28
c) Varios namespaces en el mismo documento....28
4. Namespaces y atributos....28
5. Namespaces en la práctica (ejemplos reales)....29
6. Buenas prácticas con namespaces....29
7. Ejemplo completo con múltiples namespaces....29
8. Resumen didáctico....30
Bien formado vs. válido en XML....30
1. XML Bien Formado....30
Comparación de XML con JSON....31
1. ¿Qué es JSON?....31
2. Ejemplo paralelo de XML y JSON....31
3. Similitudes....32
4. Diferencias....32
5. Ventajas de XML....32
6. Ventajas de JSON....32
7. Ejemplo de dataset en XML vs JSON....32
8. Casos de uso....33
9. Resumen didáctico....33
Buenas prácticas al trabajar con XML....34
1. Mantener un único elemento raíz....34
2. Usar nombres descriptivos y consistentes....34
3. Documentar con comentarios....34
4. Usar atributos y elementos de forma adecuada....35
5. Indentar y formatear correctamente....35
6. Usar codificación UTF-8....35
7. Validar los documentos....35
8. Evitar redundancias y datos innecesarios....36
9. Manejar namespaces correctamente....36
10. Pensar en el tamaño y rendimiento....36
11. Usar herramientas adecuadas....36
12. Mantener la compatibilidad....37
13. Resumen didáctico....37
Como leer un xml desde python....37
1. Ejemplo de lectura de un archivo xml desde lenguaje python....37

¿Qué es XML?

Definición

XML (eXtensible Markup Language) es un lenguaje de marcado diseñado para almacenar, estructurar e intercambiar datos de forma legible tanto para humanos como para máquinas.

No es un lenguaje de programación, sino un formato de datos.

Se basa en etiquetas (similares a HTML), pero a diferencia de este, no tiene etiquetas predefinidas: el desarrollador puede crear sus propias etiquetas para describir la información de manera flexible.

Ejemplo de un archivo XML simple:

```
<?xml version="1.0" encoding="UTF-8"?>
<persona>
    <nombre>María</nombre>
    <edad>30</edad>
    <ciudad>Córdoba</ciudad>
</persona>
```

En este ejemplo:

- `<persona>` es el elemento **raíz**.
- Contiene otros elementos (`<nombre>`, `<edad>`, `<ciudad>`), que representan información estructurada.

Características principales

- **Extensible**: se pueden crear etiquetas personalizadas según la necesidad.
- **Legible**: es texto plano, entendible por humanos y máquinas.
- **Jerárquico**: organiza la información en forma de árbol (elementos dentro de otros).
- **Independiente de plataforma**: puede usarse en cualquier sistema operativo o aplicación.
- **Interoperable**: se utiliza como formato estándar para intercambiar datos entre sistemas diferentes.

¿Dónde se utiliza XML actualmente?

Aunque JSON ha ganado mucho protagonismo en aplicaciones web modernas, XML sigue siendo muy utilizado en diferentes ámbitos:

Configuración de aplicaciones

- Archivos como `web.config`, `app.config` en .NET.
- Configuración de Android (AndroidManifest.xml).
- Configuración de Maven en Java (`pom.xml`).

Intercambio de datos en sistemas empresariales

- Protocolos como SOAP (Web Services) aún lo usan intensivamente.
- Integración entre sistemas de bancos, seguros, logística y gobiernos.

Bases de datos

- Algunos motores permiten almacenar y consultar XML directamente (ejemplo: SQL Server, Oracle).

Estándares internacionales

- SVG (gráficos vectoriales escalables) está basado en XML.
- XHTML (versión más estricta de HTML).
- RSS/Atom para sindicación de noticias y blogs.

Documentos electrónicos

- Facturación electrónica en muchos países.
- Archivos de Microsoft Office (Word, Excel, PowerPoint modernos son básicamente ZIP con XML interno).

Industria del software y telecomunicaciones

- Archivos de configuración de servidores.
- Protocolos de mensajería como XMPP (usado en chats).

Resumen didáctico

XML es un formato estándar de datos, creado para que distintos sistemas puedan hablar el mismo idioma al intercambiar información. Aunque muchas aplicaciones modernas usan JSON por su ligereza, XML todavía es fundamental en entornos empresariales, estándares de la industria, configuraciones y documentos electrónicos.

Instalación de herramientas para trabajar con XML (editores y validadores)

Trabajar con XML no requiere programas complejos, ya que cualquier editor de texto puede abrir y guardar archivos `.xml`. Sin embargo, para aprender y trabajar de manera profesional conviene usar editores especializados y validadores que nos ayuden a detectar errores y validar la estructura de los documentos.

1. Editores de texto recomendados para XML

🖊️ Editores simples (para empezar)

- **Bloc de notas** (Windows): suficiente para archivos muy básicos, pero no ofrece ayudas visuales.
- **Gedit** (Linux): editor simple que soporta resaltado de sintaxis.

💡 Editores avanzados (recomendados)

Visual Studio Code (VS Code)

- Multiplataforma (Windows, Linux, Mac).
- Extensiones para XML (como XML Tools).
- Permite validación con DTD y XSD.
- Autocompletado de etiquetas.

Instalación:

- Descargá desde code.visualstudio.com.
- Agregá la extensión *XML Tools* desde el Marketplace.

Notepad++ (Windows)

- Ligero y rápido.
- Incluye resaltado de sintaxis para XML.
- Plugins como XML Tools permiten formatear, validar y convertir.

Instalación:

- Descargá desde notepad-plus-plus.org.
- En el menú *Plugins Admin*, buscá *XML Tools* y activalo.

Eclipse IDE (Java Developers)

- Ideal si trabajás con Java y XML (ejemplo: `pom.xml` de Maven).
- Incluye validación y edición gráfica de XML.

Instalación:

- Descargá desde eclipse.org.

Oxygen XML Editor (Profesional, pago)

- Usado en entornos empresariales.
- Soporte completo para XML, XSD, DTD, XSLT, XPath, XQuery.
- Tiene versión de prueba.

2. Validadores de XML

Un documento XML puede estar:

- **Bien formado**: cumple con las reglas básicas de sintaxis XML.
- **Válido**: además respeta un DTD o XSD definido.

Los validadores sirven para comprobar estas reglas.

🔍 Validadores online

W3C Validator para XML
Sitio: validator.w3.org
Permite pegar el código XML o subir el archivo. Revisa sintaxis y validez.

XML Validation (FreeFormatter)
Sitio: freeformatter.com/xml-validator-xsd.html
Compatible con DTD y XSD.

XML Lint (Online)
Permite validar rápidamente si el documento es bien formado.

✅ Validadores locales (línea de comandos)

xmllint (Linux/Mac/Windows con Cygwin)

- Herramienta de línea de comandos incluida en libxml2.

Ejemplo de uso:

```
xmllint archivo.xml --noout
# indica si el XML está bien formado
```

Validación con XSD:

```
xmllint archivo.xml --schema esquema.xsd --noout
```

Xerces (Java)

- Biblioteca de Apache que valida XML contra DTD/XSD.
- Se usa mucho en entornos empresariales.

3. Instalación práctica paso a paso (Windows)

Instalar Visual Studio Code

1. Descargar e instalar desde la web oficial.
2. Abrir VS Code.
3. Ir a Extensiones (`Ctrl+Shift+X`).
4. Buscar *XML Tools* → Instalar.

Probar con un archivo XML simple

Crear un archivo `ejemplo.xml`:

```
<?xml version="1.0" encoding="UTF-8"?>
<libro>
    <titulo>El Quijote</titulo>
    <autor>Miguel de Cervantes</autor>
    <año>1605</año>
</libro>
```

Guardar el archivo y usar la extensión para formatear y validar.

Validar online

1. Copiar el contenido y pegarlo en el W3C Validator.
2. Verificar que el archivo esté bien formado.

4. Recomendación didáctica

👉 Para este curso, recomiendo usar **VS Code** o **Notepad++**, ya que:

- Son gratuitos y fáciles de instalar.
- Permiten validar y formatear XML sin complicaciones.
- Se integran con otros lenguajes y frameworks (como Python, Java, etc.).

Estructura básica de un documento XML

Un archivo XML es un documento de texto plano que debe cumplir ciertas reglas para ser considerado bien formado. La estructura general de cualquier documento XML se compone de:

- Prólogo (declaración inicial)
- Elemento raíz (*root element*)
- Elementos hijos y contenido
- Atributos (opcional)
- Comentarios (opcional)

Veamos cada parte en detalle con ejemplos.

1. Prólogo (declaración inicial)

El prólogo es opcional, pero recomendado. Generalmente se incluye la declaración XML:

```
<?xml version="1.0" encoding="UTF-8"?>
```

- `version="1.0"`: versión del estándar XML.
- `encoding="UTF-8"`: define la codificación de caracteres (UTF-8 es la más usada).

Si no se especifica, algunos programas pueden interpretar mal caracteres especiales (acentos, ñ, símbolos).

2. Elemento raíz

Todo documento XML debe tener un único elemento raíz que contenga a todos los demás elementos.

Ejemplo válido:

```
<libro>
    <titulo>Don Quijote</titulo>
    <autor>Miguel de Cervantes</autor>
</libro>
```

Ejemplo inválido (dos elementos raíz):

```
<libro>Don Quijote</libro>
<libro>La Odisea</libro>
```

Esto generará un error porque solo puede existir un *root element*.

3. Elementos hijos y contenido

Los elementos se representan con etiquetas de apertura y cierre:

```
<titulo>Don Quijote</titulo>
```

- `<titulo>`: etiqueta de apertura.
- `Don Quijote`: contenido del elemento.
- `</titulo>`: etiqueta de cierre.

Regla importante: las etiquetas deben cerrarse siempre.

Ejemplo correcto:

```
<autor>Miguel de Cervantes</autor>
```

Ejemplo incorrecto:

```
<autor>Miguel de Cervantes
```

4. Atributos

Los elementos pueden tener atributos, que aportan información adicional.

```
<libro id="1" idioma="es">
    <titulo>Don Quijote</titulo>
    <autor>Miguel de Cervantes</autor>
</libro>
```

- `id="1"` e `idioma="es"` son atributos.
- Los valores de atributos siempre deben ir entre comillas dobles o simples.

Diferencia:

- Un **atributo** describe una característica.
- Un **elemento** guarda datos.

Ejemplo ilustrativo:

```
<!-- Con atributos -->
<persona nombre="Juan" edad="30"/>

<!-- Con elementos -->
<persona>
    <nombre>Juan</nombre>
    <edad>30</edad>
</persona>
```

Ambas formas son válidas, pero depende del contexto elegir una u otra.

5. Comentarios

Los comentarios ayudan a documentar el XML y no afectan su procesamiento:

```
<!-- Este es un comentario -->
<persona>
    <nombre>Ana</nombre>
</persona>
```

Regla: no se permite `--` dentro de un comentario.

6. Ejemplo completo de estructura básica

```
<?xml version="1.0" encoding="UTF-8"?>

<!-- Ejemplo de un documento XML -->
<biblioteca>
    <libro id="1" idioma="es">
        <titulo>Don Quijote</titulo>
        <autor>Miguel de Cervantes</autor>
        <año>1605</año>
    </libro>
    <libro id="2" idioma="gr">
        <titulo>La Odisea</titulo>
        <autor>Homero</autor>
        <año>-700</año>
    </libro>
</biblioteca>
```

Análisis:

- Prólogo con versión y codificación.
- Comentario inicial.
- `<biblioteca>` como elemento raíz.
- Varios `<libro>` como elementos hijos.
- Cada libro tiene elementos (`titulo`, `autor`, `año`) y atributos (`id`, `idioma`).

7. Reglas clave para un XML bien formado

- Un único elemento raíz.
- Las etiquetas deben cerrarse siempre.
- Los elementos deben estar correctamente anidados (no se pueden cruzar).

Ejemplo inválido:

```
<persona>
    <nombre>Juan
    <apellido>Pérez</nombre>
</apellido>
</persona>
```

Corrección:

```
<persona>
    <nombre>Juan</nombre>
    <apellido>Pérez</apellido>
</persona>
```

- Los atributos siempre con valores entre comillas.
- Diferencia entre mayúsculas y minúsculas (XML es *case-sensitive*).

Ejemplo: `<Nombre>` y `<nombre>` son elementos distintos.

8. Resumen didáctico

La estructura básica de un documento XML se puede resumir así:

```
<?xml version="1.0" encoding="UTF-8"?>
<raiz>
    <elemento atributo="valor">Contenido</elemento>
    <elemento>Más contenido</elemento>
</raiz>
```

Si se cumplen estas reglas, el documento será considerado bien formado y podrá ser procesado por cualquier herramienta XML.

Elementos y etiquetas en XML

Los elementos son las unidades básicas de un documento XML. Se representan mediante etiquetas de apertura y cierre, y pueden contener:

- Texto
- Otros elementos (hijos)
- Atributos

1. ¿Qué es una etiqueta en XML?

Una etiqueta es la forma en que definimos un elemento:

```
<nombre>Juan</nombre>
```

- `<nombre>`: etiqueta de apertura.
- `Juan`: contenido.
- `</nombre>`: etiqueta de cierre.

Con esto se forma un elemento completo: `<nombre>Juan</nombre>`.

2. Tipos de elementos

a) Elementos simples

Contienen solo texto:

```
<ciudad>Córdoba</ciudad>
```

b) Elementos con hijos (anidados)

Un elemento puede contener otros elementos:

```
<persona>
    <nombre>Juan</nombre>
    <apellido>Pérez</apellido>
</persona>
```

c) Elementos vacíos (self-closing)

No tienen contenido, solo sirven como marcadores:

```
<linea />
```

Estos son muy comunes en documentos que representan estructura o configuración.

3. Nombres de elementos

Los nombres de elementos (etiquetas) deben seguir estas reglas:

- No pueden comenzar con un número.
- No pueden contener espacios.
- No pueden usar caracteres especiales (`!`, `?`, `%`, `&`, etc.).
- Son sensibles a mayúsculas/minúsculas (`<Nombre>` ≠ `<nombre>`).
- Es recomendable usar nombres descriptivos.

Correcto:

```
<producto>Computadora</producto>
```

Incorrecto:

```
<1producto>Computadora</1producto>   <!-- No puede iniciar con número -->
<producto nombre>Computadora</producto nombre> <!-- No se permiten espacios -->
```

4. Jerarquía de elementos

Los elementos en XML forman una estructura de árbol. Un documento tiene un único elemento raíz. Dentro de él, hay elementos hijos, nietos, etc.

Ejemplo jerárquico:

```
<biblioteca>
    <libro>
        <titulo>Don Quijote</titulo>
        <autor>Miguel de Cervantes</autor>
    </libro>
    <libro>
        <titulo>La Odisea</titulo>
        <autor>Homero</autor>
    </libro>
</biblioteca>
```

Estructura en árbol:

```
biblioteca
  libro
      titulo
      autor
  libro
      titulo
      autor
```

5. Atributos vs. elementos

Un atributo agrega información adicional dentro de una etiqueta, mientras que un elemento guarda datos.

Ejemplo con atributos:

```
<libro id="1" idioma="es">
    <titulo>Don Quijote</titulo>
</libro>
```

Ejemplo con elementos:

```
<libro>
    <id>1</id>
    <idioma>es</idioma>
    <titulo>Don Quijote</titulo>
</libro>
```

Ambos son válidos. La elección depende de la convención:

- Si es información secundaria, usar **atributos**.
- Si es información principal, usar **elementos**.

6. Ejemplo completo

```
<?xml version="1.0" encoding="UTF-8"?>

<empresa>
    <empleado id="101" departamento="Ventas">
        <nombre>Laura</nombre>
        <apellido>Gómez</apellido>
        <edad>28</edad>
    </empleado>
    <empleado id="102" departamento="TI">
        <nombre>Carlos</nombre>
        <apellido>Pérez</apellido>
        <edad>35</edad>
    </empleado>
</empresa>
```

Análisis:

- `<empresa>` es el elemento raíz.
- Contiene varios `<empleado>` como hijos.
- Cada `<empleado>` tiene elementos internos (`nombre`, `apellido`, `edad`).
- Los atributos (`id`, `departamento`) complementan la información.

7. Errores comunes con elementos y etiquetas

No cerrar una etiqueta

```
<nombre>Juan
```

Incorrecto.

Correcto:

```
<nombre>Juan</nombre>
```

Cruzar etiquetas (anidamiento incorrecto)

```
<persona>
    <nombre>Juan
    <apellido>Pérez</nombre>
</apellido>
</persona>
```

Incorrecto.

Correcto:

```
<persona>
```

```
    <nombre>Juan</nombre>
    <apellido>Pérez</apellido>
</persona>
```

Nombres inválidos en etiquetas

```
<123nombre>Juan</123nombre>   <!-- No válido -->
```

8. Resumen didáctico

- Los elementos XML son bloques de datos delimitados por etiquetas.
- Existen elementos simples, compuestos y vacíos.
- Los nombres deben ser válidos y descriptivos.
- La jerarquía de elementos forma un árbol XML.
- Los atributos complementan a los elementos, pero no deben usarse para información principal.

Con esta base, ya se puede modelar cualquier tipo de información en XML de manera clara y estructurada.

Atributos en XML

Los atributos en XML son pares *nombre-valor* que se colocan dentro de la etiqueta de apertura de un elemento. Su función es agregar información adicional a ese elemento, sin formar parte del contenido principal.

Ejemplo básico

```
<libro id="1" idioma="es">
    <titulo>Don Quijote</titulo>
    <autor>Miguel de Cervantes</autor>
</libro>
```

Aquí:

- `id="1"` e `idioma="es"` son atributos del elemento `<libro>`.
- El contenido principal (`<titulo>` y `<autor>`) sigue siendo representado por elementos hijos.

1. Reglas para los atributos

- Siempre se escriben en la etiqueta de apertura de un elemento.
- Sus valores deben estar entre comillas dobles o simples:

```
<persona nombre="Ana"/>
<persona nombre='Ana'/>
```

Un elemento puede tener varios atributos:

```
<empleado id="1001" cargo="analista" area="TI"/>
```

Los nombres de atributos deben seguir las mismas reglas que los elementos:

- No comenzar con números.
- No usar espacios ni caracteres especiales.
- Son *case-sensitive* (distinguen mayúsculas y minúsculas):

```
<persona nombre="Juan"/>   <!-- válido -->
<persona Nombre="Juan"/>   <!-- es un atributo distinto -->
```

2. Diferencia entre atributos y elementos

Una de las dudas más comunes es: ¿cuándo uso un atributo y cuándo un elemento?

Ejemplo con atributo:

```
<persona nombre="Laura" edad="30"/>
```

Ejemplo con elementos:

```
<persona>
    <nombre>Laura</nombre>
    <edad>30</edad>
</persona>
```

Ambos son válidos, pero la elección depende del contexto:

- **Atributos**: usados para propiedades secundarias o metadatos.
- **Elementos**: usados para datos principales o que puedan crecer en complejidad.

Regla práctica:

- Si es parte esencial de la información, usá **elementos**.
- Si es información extra que describe al elemento, usá **atributos**.

3. Ejemplos prácticos

a) Información de un libro

Con atributos:

```
<libro id="1" idioma="es" año="1605">
    <titulo>Don Quijote</titulo>
    <autor>Miguel de Cervantes</autor>
</libro>
```

Con elementos:

```
<libro>
    <id>1</id>
```

```
    <idioma>es</idioma>
    <año>1605</año>
    <titulo>Don Quijote</titulo>
    <autor>Miguel de Cervantes</autor>
</libro>
```

Ambos representan lo mismo, pero el primero es más ligero y fácil de procesar cuando los atributos son pocos.

b) Datos de un usuario

Uso mixto (recomendado):

```
<usuario id="501" activo="true">
    <nombre>Carla</nombre>
    <email>carla@example.com</email>
</usuario>
```

- `id` y `activo` son atributos porque describen al usuario.
- `nombre` y `email` son elementos, porque son datos principales.

4. Casos donde conviene usar atributos

- Identificadores (`id`, `codigo`).
- Estados o banderas (`activo="true"`, `disponible="no"`).
- Propiedades simples (`idioma`, `versión`, `formato`).

Ejemplo:

```
<imagen id="img001" formato="jpg" ancho="800" alto="600"/>
```

5. Casos donde conviene usar elementos

- Datos que pueden repetirse o ser complejos.
- Contenido principal que no es accesorio.
- Información jerárquica o estructurada.

Ejemplo (dirección de una persona):

```
<direccion>
    <calle>Av. Colón</calle>
    <numero>32</numero>
    <ciudad>Córdoba</ciudad>
</direccion>
```

No sería buena práctica meter todo en un atributo como:

```
<direccion datos="Av. Colón, 32, Córdoba"/>
```

< "/>

6. Ejemplo completo con atributos y elementos

```
<?xml version="1.0" encoding="UTF-8"?>

<biblioteca>
    <libro id="001" idioma="es" disponible="true">
        <titulo>Don Quijote</titulo>
        <autor>Miguel de Cervantes</autor>
        <genero>Novela</genero>
    </libro>
    <libro id="002" idioma="en" disponible="false">
        <titulo>Hamlet</titulo>
        <autor>William Shakespeare</autor>
        <genero>Teatro</genero>
    </libro>
</biblioteca>
```

Análisis:

- Los atributos (`id`, `idioma`, `disponible`) describen a cada libro.
- Los elementos (`titulo`, `autor`, `genero`) contienen la información principal.

7. Errores comunes con atributos

Olvidar las comillas

```
<persona nombre=Juan/>  <!-- Incorrecto -->
<persona nombre="Juan"/> <!-- Correcto -->
```

Usar el mismo atributo dos veces en un mismo elemento

```
<persona nombre="Juan" nombre="Pedro"/> <!-- Inválido -->
```

Guardar datos complejos en atributos (mala práctica)

```
<pedido productos="Leche,Pan,Huevos"/> <!-- Difícil de procesar -->
```

Mejor con elementos:

```
<pedido>
    <producto>Leche</producto>
    <producto>Pan</producto>
    <producto>Huevos</producto>
</pedido>
```

8. Resumen didáctico

- Los atributos agregan información adicional dentro de una etiqueta.
- Son útiles para identificadores, estados o propiedades simples.
- Los elementos son mejores para datos principales y estructurados.

- Una buena práctica es combinar ambos, usando atributos para describir y elementos para contener información.

Dominar este equilibrio es fundamental para diseñar documentos XML claros, fáciles de leer y procesar.

Declaración XML y codificación de caracteres

1. ¿Qué es la declaración XML?

La declaración de XML es una línea opcional (pero recomendada) que aparece al inicio de un archivo. Sirve para indicar:

- La versión del estándar XML usada.
- La codificación de caracteres (*encoding*).
- (Opcional) si el documento es independiente de un DTD externo (`standalone`).

Ejemplo típico:

```
<?xml version="1.0" encoding="UTF-8"?>
```

2. Atributos de la declaración XML

a) version

- Define la versión del estándar XML.
- La más usada es `1.0` (compatible en casi todos los sistemas).
- Existe `1.1`, pero su uso es poco común.

```
<?xml version="1.0"?>
```

b) encoding

Define la codificación de caracteres usada en el documento. Esto indica cómo se representan letras, símbolos y caracteres especiales.

Los más usados son:

- **UTF-8**: estándar universal, soporta todos los idiomas y emojis.
- **ISO-8859-1 (Latin-1)**: soporta caracteres occidentales (acentos, ñ), pero no otros alfabetos.
- **UTF-16**: soporta caracteres extendidos, aunque suele ser menos eficiente en tamaño.

Ejemplo con ISO-8859-1:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
```

Ejemplo con UTF-16:

```
<?xml version="1.0" encoding="UTF-16"?>
```

Si no se indica `encoding`, por defecto se asume UTF-8.

c) standalone

Indica si el documento depende de un archivo externo (como un DTD) para validarse.

- `standalone="yes"`: el documento es independiente.
- `standalone="no"`: el documento necesita un archivo externo.

Ejemplo:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
```

3. Importancia de la codificación

Si no se define correctamente la codificación, el documento puede mostrar errores en caracteres especiales:

Ejemplo con UTF-8 (correcto):

```
<?xml version="1.0" encoding="UTF-8"?>
<mensaje>
    Hola, señor García! 😀
</mensaje>
```

Ejemplo con ISO-8859-1 (se rompe con emoji):

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<mensaje>
    Hola, señor García! 😀
</mensaje>
```

Resultado: la letra *ñ* y los acentos funcionan, pero el emoji dará error porque ISO-8859-1 no lo soporta.

4. Ejemplos comparativos

a) Documento en UTF-8

```
<?xml version="1.0" encoding="UTF-8"?>
<ciudades>
    <ciudad>São Paulo</ciudad>
    <ciudad>München</ciudad>
    <ciudad>北京</ciudad>
    <ciudad>東京</ciudad>
</ciudades>
```

UTF-8 permite representar caracteres latinos, chinos, japoneses y acentos sin problema.

b) Documento en ISO-8859-1

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<ciudades>
    <ciudad>São Paulo</ciudad>
    <ciudad>München</ciudad>
    <ciudad>北京</ciudad> <!-- ERROR: no soportado -->
</ciudades>
```

ISO-8859-1 solo soporta caracteres occidentales, por eso falla con chino o japonés.

5. Buenas prácticas

- Siempre incluir la declaración XML al inicio.
- Usar **UTF-8** como estándar universal (recomendado por el W3C).
- Evitar mezclar distintas codificaciones en el mismo sistema.
- Validar el archivo en un editor (por ejemplo, VS Code, Notepad++) para detectar errores de codificación.

6. Ejemplo completo

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>

<!-- Ejemplo de documento XML con caracteres internacionales -->
<mensajes>
    <mensaje id="1">Hola, señor García!</mensaje>
    <mensaje id="2">Привет, мир</mensaje>
    <mensaje id="3">こんにちは世界</mensaje>
    <mensaje id="4">مرحبا بالعالم</mensaje>
</mensajes>
```

Gracias a UTF-8, este documento soporta español, ruso, japonés y árabe en el mismo archivo.

7. Resumen didáctico

La declaración XML se coloca al inicio del archivo y define:

- Versión (`version="1.0"`)
- Codificación (`encoding="UTF-8"`)
- Independencia (`standalone="yes/no"`)

UTF-8 es la codificación recomendada porque soporta todos los idiomas y símbolos. Una codificación incorrecta puede romper el archivo y generar errores en lectores XML.

Comentarios en XML

Los comentarios en XML permiten agregar notas, explicaciones o documentación interna dentro del archivo, sin que estas afecten al contenido ni a la validación del documento.

Sirven para:

- Explicar la finalidad de ciertos elementos o atributos.
- Marcar secciones del documento.
- Dejar recordatorios o notas para otros desarrolladores.

1. Sintaxis de un comentario

Un comentario en XML se escribe así:

```
<!-- Este es un comentario -->
```

Reglas básicas:

- Comienza con `<!--` y termina con `-->`.
- No puede contener `--` en su interior (es inválido).
- Puede estar en cualquier parte del documento, salvo dentro de la declaración XML.

2. Ejemplos de comentarios

a) Comentario simple

```
<!-- Lista de libros de la biblioteca -->
<biblioteca>
    <libro>Don Quijote</libro>
    <libro>La Odisea</libro>
</biblioteca>
```

b) Comentario en una línea dentro del contenido

```
<biblioteca>
    <libro>Don Quijote</libro> <!-- Obra española -->
    <libro>La Odisea</libro>   <!-- Obra griega -->
</biblioteca>
```

c) Comentarios multilínea

```
<!--
    Archivo XML de ejemplo.
    Contiene datos de empleados.
    Cada empleado tiene: id, nombre y área.
-->
<empresa>
    <empleado id="1">
        <nombre>Laura</nombre>
        <area>Ventas</area>
    </empleado>
</empresa>
```

3. Dónde se pueden colocar

Lugares permitidos:

- Antes del elemento raíz.
- Dentro del elemento raíz, entre elementos.
- Al final del documento.

Lugares no permitidos:

- Dentro de la declaración XML:

```
<?xml version="1.0" encoding="UTF-8"?> <!-- Inválido aquí -->
```

4. Buenas prácticas con comentarios

- Usarlos para explicar el contenido, no para guardar información que debería estar en el XML.
- Mantenerlos claros y breves.
- Colocarlos en lugares estratégicos (antes de secciones importantes).
- Usarlos para dividir bloques grandes de XML:

```
<!-- ======================
     Sección de clientes
====================== -->
<clientes>
    <cliente id="101">
        <nombre>Juan</nombre>
        <ciudad>Córdoba</ciudad>
    </cliente>
</clientes>
```

Documentar estructuras complejas:

```
<!--
   El elemento "pedido" contiene:
   - id (atributo)
   - fecha (atributo)
   - productos (elementos hijos)
-->
<pedido id="5001" fecha="2025-09-09">
    <producto>Computadora</producto>
    <producto>Mouse</producto>
</pedido>
```

5. Malas prácticas con comentarios

- Usar comentarios para desactivar información en producción de forma permanente.

```
<!-- <usuario activo="false"/> -->
```

Esto puede generar confusión si alguien piensa que el elemento nunca existió.

- Guardar datos importantes en comentarios.

```
<!-- La contraseña es 1234 -->
```

Nunca se deben dejar credenciales o información sensible en un comentario.

- Uso excesivo que ensucia el archivo.

Los comentarios son útiles, pero si hay demasiados, dificultan la lectura.

6. Ejemplo completo con comentarios

```
<?xml version="1.0" encoding="UTF-8"?>

<!--
    Ejemplo de archivo XML con comentarios
    Curso de XML - Tema 7
-->

<biblioteca>
    <!-- Primer libro en español -->
    <libro id="1" idioma="es">
        <titulo>Don Quijote</titulo>
        <autor>Miguel de Cervantes</autor>
    </libro>

    <!-- Segundo libro, clásico griego -->
    <libro id="2" idioma="gr">
        <titulo>La Odisea</titulo>
        <autor>Homero</autor>
    </libro>
</biblioteca>
```

7. Resumen didáctico

- Los comentarios se escriben con `<!-- ... -->`.
- No pueden contener `--` en su interior.
- Sirven para documentar y explicar partes del XML.
- Buenas prácticas: claros, breves y estratégicos.
- Malas prácticas: usarlos como "basurero" de datos o llenarlos de información innecesaria.

Con comentarios bien usados, un archivo XML se vuelve más legible y mantenible, sobre todo en proyectos grandes con varios desarrolladores.

Jerarquía y anidamiento de elementos en XML

1. Concepto de jerarquía en XML

Un documento XML organiza sus elementos en forma de árbol jerárquico:

- En la cima está el elemento **raíz**.
- Dentro de él, se encuentran elementos **hijos**.
- Estos hijos pueden tener **nietos**, y así sucesivamente.
- Los elementos que comparten el mismo padre son **hermanos**.

Esta estructura en árbol hace que XML sea muy intuitivo y potente para representar datos complejos.

2. Definiciones clave

- **Elemento raíz (root element)**: el único elemento que contiene a todos los demás.
- **Padre**: un elemento que contiene a otros elementos dentro de sí.
- **Hijo**: un elemento que está contenido dentro de otro.
- **Nieto**: un elemento que está contenido dentro de un hijo.
- **Hermanos (siblings)**: elementos que comparten el mismo padre.

3. Ejemplo básico de jerarquía

```
<biblioteca>
    <libro>
        <titulo>Don Quijote</titulo>
        <autor>Miguel de Cervantes</autor>
    </libro>
    <libro>
        <titulo>La Odisea</titulo>
        <autor>Homero</autor>
    </libro>
</biblioteca>
```

Análisis de la jerarquía:

- `<biblioteca>` es el elemento raíz.
- `<libro>` es hijo de `<biblioteca>`.
- `<titulo>` y `<autor>` son hijos de `<libro>` y nietos de `<biblioteca>`.
- Los dos `<libro>` son hermanos porque comparten el mismo padre (`<biblioteca>`).

4. Representación en forma de árbol

El ejemplo anterior se puede ver así:

```
biblioteca
  libro
      titulo
      autor
  libro
      titulo
      autor
```

Esta representación en árbol refleja la estructura jerárquica natural de XML.

5. Reglas de anidamiento correcto

- Todos los elementos deben cerrarse en orden inverso a su apertura (no se pueden cruzar).

Ejemplo incorrecto (etiquetas cruzadas):

```
<persona>
    <nombre>Juan
    <apellido>Pérez</nombre>
</apellido>
</persona>
```

Ejemplo correcto:

```
<persona>
    <nombre>Juan</nombre>
    <apellido>Pérez</apellido>
</persona>
```

6. Ejemplo de jerarquía más compleja

```
<empresa>
    <departamento id="D1">
        <nombre>Ventas</nombre>
        <empleado id="101">
            <nombre>Laura</nombre>
            <puesto>Vendedora</puesto>
        </empleado>
        <empleado id="102">
            <nombre>Carlos</nombre>
            <puesto>Supervisor</puesto>
        </empleado>
    </departamento>
    <departamento id="D2">
        <nombre>TI</nombre>
        <empleado id="201">
            <nombre>Ana</nombre>
            <puesto>Programadora</puesto>
        </empleado>
    </departamento>
</empresa>
```

Jerarquía:

- `<empresa>`: raíz.

- `<departamento>`: hijos de `<empresa>`.
- `<empleado>`: hijos de `<departamento>`.
- `<nombre>` y `<puesto>`: hijos de `<empleado>` y nietos de `<departamento>`.
- Los `<departamento>` son hermanos entre sí.

7. Ejemplo con atributos + elementos anidados

```
<universidad>
    <facultad nombre="Ingeniería">
        <carrera nombre="Informática">
            <materia codigo="101">Programación I</materia>
            <materia codigo="102">Bases de Datos</materia>
        </carrera>
        <carrera nombre="Electrónica">
            <materia codigo="201">Circuitos</materia>
        </carrera>
    </facultad>
</universidad>
```

Estructura en árbol:

```
universidad
  facultad (nombre=Ingeniería)
       carrera (nombre=Informática)
           materia (codigo=101)
           materia (codigo=102)
       carrera (nombre=Electrónica)
            materia (codigo=201)
```

8. Buenas prácticas en jerarquía y anidamiento

- Usar nombres descriptivos en los elementos.
- Mantener una estructura lógica y coherente (de lo general a lo específico).
- Evitar anidamientos excesivos que dificulten la lectura.
- Combinar atributos y elementos de manera adecuada: atributos para propiedades, elementos para datos principales.
- Mantener un indentado consistente para facilitar la lectura.

9. Ejemplo de mala práctica

```
<empresa>
    <departamento id="D1" empleado1="Laura,Vendedora"
empleado2="Carlos,Supervisor"/>
</empresa>
```

Aunque es válido técnicamente, es difícil de leer y procesar.

Mejor con elementos anidados:

```
<empresa>
    <departamento id="D1">
```

```
        <empleado>
            <nombre>Laura</nombre>
            <puesto>Vendedora</puesto>
        </empleado>
        <empleado>
            <nombre>Carlos</nombre>
            <puesto>Supervisor</puesto>
        </empleado>
    </departamento>
</empresa>
```

10. Resumen didáctico

XML organiza sus datos en una jerarquía de elementos en forma de árbol.

Conceptos:

- **Padre**: contiene elementos.
- **Hijo**: está dentro de otro.
- **Hermano**: comparte el mismo padre.

Reglas: no cruzar etiquetas y siempre cerrarlas correctamente.

La jerarquía clara y el buen anidamiento hacen que un XML sea legible, mantenible y fácil de procesar.

Espacios de nombres (Namespaces) en XML

1. ¿Qué es un namespace en XML?

Un espacio de nombres (*namespace*) es una forma de identificar de manera única los elementos y atributos dentro de un documento XML. Se utiliza principalmente para evitar conflictos de nombres cuando se combinan datos de diferentes vocabularios o estándares.

Se define con un atributo especial `xmlns` dentro de un elemento.

Ejemplo básico (namespace por defecto):

```
<libro xmlns="http://ejemplo.com/biblioteca">
    <titulo>Don Quijote</titulo>
    <autor>Miguel de Cervantes</autor>
</libro>
```

Aquí `xmlns="http://ejemplo.com/biblioteca"` define el namespace por defecto. Todos los elementos dentro de `<libro>` pertenecen a ese espacio de nombres.

2. Problema que resuelven los namespaces

Imaginemos que queremos unir información de dos sistemas distintos, cada uno con su propia definición de `<titulo>`.

Ejemplo sin namespaces (conflicto):

```
<documento>
    <titulo>Don Quijote</titulo>   <!-- ¿Título de un libro? -->
    <titulo>Lic. en Informática</titulo> <!-- ¿Título académico? -->
</documento>
```

El procesador XML no puede distinguir a qué tipo de título se refiere cada etiqueta.

Con namespaces:

```
<documento
    xmlns:lib="http://ejemplo.com/libros"
    xmlns:uni="http://ejemplo.com/universidad">

    <lib:titulo>Don Quijote</lib:titulo>
    <uni:titulo>Lic. en Informática</uni:titulo>
</documento>
```

Ahora queda claro: `lib:titulo` pertenece al namespace de libros y `uni:titulo` al de universidad.

3. Sintaxis de namespaces

a) Declaración por defecto

Todos los elementos dentro de un nodo heredan el mismo namespace.

```
<biblioteca xmlns="http://ejemplo.com/biblioteca">
    <libro>
        <titulo>La Odisea</titulo>
    </libro>
</biblioteca>
```

Aquí todos los elementos pertenecen a `http://ejemplo.com/biblioteca`.

b) Declaración con prefijo

Se usa un prefijo para identificar un namespace.

```
<biblioteca xmlns:lib="http://ejemplo.com/libros">
    <lib:libro>
        <lib:titulo>Hamlet</lib:titulo>
    </lib:libro>
</biblioteca>
```

`xmlns:lib="http://ejemplo.com/libros"` define el namespace con prefijo `lib`. Ahora cada elemento debe usar el prefijo `lib:`.

c) Varios namespaces en el mismo documento

```
<documento
    xmlns:lib="http://ejemplo.com/libros"
    xmlns:uni="http://ejemplo.com/universidad">

    <lib:libro>
        <lib:titulo>Don Quijote</lib:titulo>
    </lib:libro>

    <uni:alumno>
        <uni:nombre>Laura</uni:nombre>
        <uni:titulo>Lic. en Matemática</uni:titulo>
    </uni:alumno>
</documento>
```

Aquí combinamos datos de libros y de universidad en un solo XML sin conflictos.

4. Namespaces y atributos

Los atributos también pueden pertenecer a namespaces:

```
<empleado xmlns:info="http://ejemplo.com/empleados" info:id="1001">
    <nombre>Carlos</nombre>
</empleado>
```

Aquí el atributo `id` pertenece al namespace `info`.

5. Namespaces en la práctica (ejemplos reales)

- XHTML usa namespaces para diferenciar elementos de HTML.
- SOAP (protocolos de servicios web) define sus mensajes XML con namespaces para separar encabezados y cuerpo.

Ejemplo SOAP (simplificado):

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Header>
        <!-- Encabezado de la solicitud -->
    </soap:Header>
    <soap:Body>
        <!-- Datos de la operación -->
    </soap:Body>
</soap:Envelope>
```

Los elementos `soap:Envelope`, `soap:Header` y `soap:Body` pertenecen al namespace SOAP.

6. Buenas prácticas con namespaces

- Usar URIs únicas como identificadores de namespace (no tienen que ser enlaces reales).
- Usar prefijos cortos y descriptivos (`lib`, `uni`, `soap`).

- Evitar el uso excesivo de namespaces si no es necesario.
- Documentar qué namespace corresponde a cada prefijo.

7. Ejemplo completo con múltiples namespaces

```
<?xml version="1.0" encoding="UTF-8"?>

<registro
    xmlns:lib="http://ejemplo.com/libros"
    xmlns:per="http://ejemplo.com/personas">

    <!-- Información de un libro -->
    <lib:libro>
        <lib:titulo>1984</lib:titulo>
        <lib:autor>George Orwell</lib:autor>
    </lib:libro>

    <!-- Información de una persona -->
    <per:persona>
        <per:nombre>Ana</per:nombre>
        <per:apellido>Pérez</per:apellido>
    </per:persona>
</registro>
```

Aquí: los elementos relacionados con libros (libro, título, autor) están en el namespace `lib`, y los de persona (nombre, apellido) en el namespace `per`.

8. Resumen didáctico

- Los namespaces en XML evitan conflictos de nombres cuando se combinan datos de distintos orígenes.
- Se definen con el atributo `xmlns` (por defecto o con prefijo).
- Se aplican tanto a elementos como a atributos.
- Son fundamentales en estándares modernos como XHTML, SOAP y SVG.

Dominar namespaces es esencial para trabajar con XML en proyectos grandes e integraciones de sistemas.

Bien formado vs. válido en XML

En XML existen dos conceptos fundamentales para evaluar un documento:

- **Bien formado (well-formed)**: cumple las reglas básicas de sintaxis de XML.
- **Válido (valid)**: además de estar bien formado, respeta un conjunto de reglas definidas en un DTD o XML Schema (XSD).

Todo documento válido es necesariamente bien formado, pero no todo documento bien formado es válido.

1. XML Bien Formado

Un documento XML es bien formado si cumple con las reglas básicas de sintaxis:

- Tiene un único elemento raíz.
- Todas las etiquetas se abren y cierran correctamente.
- Los elementos están correctamente anidados (no cruzados).
- Los atributos tienen valores entre comillas.
- Diferencia entre mayúsculas y minúsculas (`<Nombre>` ≠ `<nombre>`).
- No se permiten caracteres inválidos.

Ejemplo de XML bien formado:

```
<?xml version="1.0" encoding="UTF-8"?>
<persona>
    <nombre>Ana</nombre>
    <apellido>Pérez</apellido>
    <edad>30</edad>
</persona>
```

Cumple todas las reglas básicas de sintaxis.

Ejemplo de XML mal formado:

```
<persona>
    <nombre>Ana
    <apellido>Pérez</nombre>
</apellido>
</persona>
```

Errores:

- Etiquetas mal anidadas (apellido se abre dentro de nombre pero se cierra fuera).
- El contenido queda inconsistente.

Comparación de XML con JSON

1. ¿Qué es JSON?

JSON (JavaScript Object Notation) es un formato de texto ligero para intercambio de datos.

- Está basado en pares *clave: valor* y estructuras como objetos y arreglos.
- Es más simple que XML, y está estrechamente ligado al mundo web y a las APIs modernas.

Ejemplo de JSON:

```
{
  "libro": {
```

```
    "id": 1,
    "titulo": "1984",
    "autor": "George Orwell",
    "año": 1949
  }
}
```

2. Ejemplo paralelo de XML y JSON

XML

```
<libro id="1">
    <titulo>1984</titulo>
    <autor>George Orwell</autor>
    <año>1949</año>
</libro>
```

JSON

```
{
  "libro": {
    "id": 1,
    "titulo": "1984",
    "autor": "George Orwell",
    "año": 1949
  }
}
```

Ambos representan lo mismo, pero JSON es más conciso.

3. Similitudes

- Ambos son formatos de texto legibles por humanos y máquinas.
- Son independientes del lenguaje (se pueden usar en Java, Python, C#/.NET, etc.).
- Admiten estructuras jerárquicas (datos anidados).
- Se usan en intercambio de datos entre aplicaciones y servicios web.

4. Diferencias

- **Sintaxis**: XML usa etiquetas (`<elemento>`); JSON usa claves y valores (`{ "clave": valor }`).
- **Verbosidad**: XML es más largo y redundante; JSON es más compacto.
- **Tipos de datos**: en XML todo es texto (los tipos se definen con XSD); JSON soporta nativamente *string*, *number*, *boolean*, *null*, arreglos y objetos.
- **Comentarios**: XML soporta (`<!-- ... -->`); JSON no soporta comentarios.
- **Orden**: en XML el orden de elementos puede ser importante; en JSON es menos relevante (se basa en claves).
- **Uso común**: XML en Web Services SOAP, configuración y documentos; JSON en APIs REST, aplicaciones web y bases NoSQL.

- **Validación**: XML con DTD/XSD (muy estricta); JSON con JSON Schema (flexible).
- **Popularidad actual**: XML muy usado en entornos empresariales; JSON predomina en la web moderna.

5. Ventajas de XML

- Autodescriptivo gracias a etiquetas.
- Permite comentarios dentro del documento.
- Excelente para documentos largos y estructurados (ej.: facturas electrónicas, contratos).
- Estándares maduros: DTD, XSD, XSLT, XPath, XQuery.
- Usado en sistemas empresariales, banca, seguros y gobierno.

6. Ventajas de JSON

- Más ligero y rápido de transmitir.
- Soporte nativo en JavaScript y muy natural en la web.
- Se integra fácilmente con APIs REST.
- Estructura más simple y menos redundante.
- Buen soporte en bases NoSQL como MongoDB y CouchDB.

7. Ejemplo de dataset en XML vs JSON

Supongamos una biblioteca con dos libros.

XML

```
<biblioteca>
    <libro id="1">
        <titulo>1984</titulo>
        <autor>George Orwell</autor>
        <año>1949</año>
    </libro>
    <libro id="2">
        <titulo>Don Quijote</titulo>
        <autor>Miguel de Cervantes</autor>
        <año>1605</año>
    </libro>
</biblioteca>
```

JSON

```
{
  "biblioteca": [
    {
      "id": 1,
      "titulo": "1984",
      "autor": "George Orwell",
      "año": 1949
    },
    {
      "id": 2,
```

```
        "titulo": "Don Quijote",
        "autor": "Miguel de Cervantes",
        "año": 1605
      }
    ]
  }
```

En JSON se ve más compacto, pero en XML la estructura jerárquica es más explícita.

8. Casos de uso

XML

- Servicios SOAP.
- Documentos legales y normativos (facturas electrónicas, contratos).
- Publicación de datos científicos, editoriales, configuraciones complejas.
- Entornos donde se requiere validación estricta.

JSON

- APIs REST (predominante en la web actual).
- Aplicaciones móviles y web modernas.
- Bases de datos NoSQL (ej.: MongoDB, CouchDB).
- Sistemas en tiempo real donde la performance es clave.

9. Resumen didáctico

- XML y JSON son formatos de datos jerárquicos muy usados en software.
- XML es más verboso, pero soporta comentarios, validación estricta y transformaciones avanzadas.
- JSON es más ligero, rápido en transmisión y el estándar de facto en la web.
- XML sigue fuerte en sectores empresariales y regulados, mientras que JSON domina en APIs modernas y desarrollo ágil.
- En muchos sistemas, ambos conviven: XML para integraciones *legacy* y JSON para nuevos servicios.

Buenas prácticas al trabajar con XML

1. Mantener un único elemento raíz

Todo documento XML debe tener un único elemento raíz. Esto asegura que el documento esté bien formado y pueda ser procesado por cualquier parser.

Incorrecto:

```
<persona>Juan</persona>
<persona>María</persona>
```

Correcto:

```
<personas>
    <persona>Juan</persona>
    <persona>María</persona>
</personas>
```

2. Usar nombres descriptivos y consistentes

- Los nombres de etiquetas deben ser claros y significativos.
- Usar una convención consistente (camelCase, snake_case o minúsculas con guiones).

Ejemplo recomendado:

```
<cliente>
    <nombreCompleto>Juan Pérez</nombreCompleto>
    <direccion>Córdoba, Argentina</direccion>
</cliente>
```

Malo:

```
<cl>
    <n>Pérez</n>
    <dir>CBA</dir>
</cl>
```

3. Documentar con comentarios

Los comentarios XML ayudan a entender la finalidad de los elementos.

```
<!-- Lista de empleados de la empresa -->
<empleados>
    <empleado id="101">
        <nombre>Ana</nombre>
        <puesto>Analista</puesto>
    </empleado>
</empleados>
```

No abuses de los comentarios ni guardes información sensible en ellos.

4. Usar atributos y elementos de forma adecuada

- **Elementos**: para datos principales.
- **Atributos**: para propiedades adicionales, identificadores o metadatos.

Ejemplo balanceado:

```
<producto id="P001" moneda="ARS">
    <nombre>Laptop</nombre>
    <precio>150000</precio>
</producto>
```

No recomendado (todo en atributos):

```
<producto id="P001" nombre="Laptop" precio="150000" moneda="ARS"/>
```

Si los valores son parte esencial del documento, mejor usarlos como elementos.

5. Indentar y formatear correctamente

La indentación mejora la legibilidad y facilita el mantenimiento.

```
<biblioteca>
    <libro>
        <titulo>1984</titulo>
        <autor>George Orwell</autor>
    </libro>
</biblioteca>
```

Sin indentación:

```
<biblioteca><libro><titulo>1984</titulo><autor>George
Orwell</autor></libro></biblioteca>
```

6. Usar codificación UTF-8

Siempre especificar la declaración XML con **UTF-8** para soportar acentos, ñ, emojis y caracteres internacionales.

```
<?xml version="1.0" encoding="UTF-8"?>
```

7. Validar los documentos

- Usar DTD o XSD para garantizar que el XML cumpla las reglas estructurales.
- Validar en cada etapa del desarrollo (editores, xmllint, validadores online).

```
xmllint --noout --schema esquema.xsd archivo.xml
```

8. Evitar redundancias y datos innecesarios

Malo (repite información):

```
<empleado>
    <id>101</id>
    <nombre>Ana</nombre>
    <nombre>Ana</nombre>
</empleado>
```

Correcto:

```
<empleado>
    <id>101</id>
    <nombre>Ana</nombre>
</empleado>
```

9. Manejar namespaces correctamente

- Usar *namespaces* (`xmlns`) cuando se combinen diferentes vocabularios XML.
- Usar prefijos cortos y descriptivos.

```
<doc xmlns:lib="http://ejemplo.com/libros"
xmlns:cli="http://ejemplo.com/clientes">
    <lib:libro>
        <lib:titulo>1984</lib:titulo>
    </lib:libro>
    <cli:cliente>
        <cli:nombre>Ana</cli:nombre>
    </cli:cliente>
</doc>
```

10. Pensar en el tamaño y rendimiento

- XML puede ser verboso: considerar compresión (gzip) al transmitir.
- Usar SAX en vez de DOM para documentos muy grandes (procesamiento en streaming).
- No incluir datos binarios directamente: usar Base64 o enlaces externos.

Ejemplo de referencia externa:

```
<imagen ruta="http://ejemplo.com/imagenes/foto.jpg"/>
```

11. Usar herramientas adecuadas

- Editores especializados: VS Code, Notepad++, Oxygen XML Editor.
- Validadores online: FreeFormatter, W3C Validator.
- Librerías en lenguajes de programación:
    - Python: xml.etree.ElementTree, lxml.
    - Java: DOM, SAX, StAX.
    - C#/.NET: `System.Xml`.

12. Mantener la compatibilidad

XML puede vivir décadas en un sistema, pensar en la evolución de versiones.

Usar atributos para versionado:

```
<factura version="1.2">
    <cliente>Juan</cliente>
</factura>
```

13. Resumen didáctico

- **Claridad**: un solo elemento raíz, nombres descriptivos, indentación.
- **Correctitud**: codificación UTF-8, validación con DTD/XSD.

- **Eficiencia**: evitar redundancias, usar compresión en transmisión.
- **Escalabilidad**: usar DOM para documentos pequeños, SAX para grandes.
- **Interoperabilidad**: namespaces bien definidos, versionado cuando sea necesario.

Seguir estas prácticas asegura que el XML sea fácil de leer, mantener, validar y compartir en cualquier entorno.

Como leer un xml desde python

1. Ejemplo de lectura de un archivo xml desde lenguaje python.

```
import xml.etree.ElementTree as ET

# Abrimos y parseamos el fichero XML
tree = ET.parse('datos.xml')

# Obtenemos el elemento raíz del fichero XML,
# es decir, la etiqueta <Data>
data = tree.getroot()

# Creamos el array vacío que contendrá los
# datos agrupados de los dependientes
lista_libros = []

# Iteramos sobre cada dependiente del fichero
for libro in data:
# Almacenamos en una lista todos los
# atributos de un dependiente
lista_atributos = [atributo.text for atributo in libro]

# Almacenamos los atributos del dependiente
# en la lista de dependientes
lista_libros.append(lista_atributos)

# Mostramos el resultado
print(lista_libros)
```